Design and Analysis of Algorithms
Greedy Algorithms (I)

1. Introduction to Greedy Algorithms
2. Interval Scheduling
3. Optimal Loading
4. Scheduling to Minimize Maximum Lateness
5. Fractional Knapsack Problem

Outline

1. Introduction to Greedy Algorithms
2. Interval Scheduling
3. Optimal Loading
4. Scheduling to Minimize Maximum Lateness
5. Fractional Knapsack Problem

Motivation

Success in a game like chess requires *thinking ahead*

- a player who focuses entirely on immediate advantages is easy to defeat.

In many other games, however, such as Scrabble

- it is reasonable to make the move that appears best at the moment without worrying too much about future consequences.





Motivation

Success in a game like chess requires *thinking ahead*

- a player who focuses entirely on immediate advantages is easy to defeat.

In many other games, however, such as Scrabble

- it is reasonable to make the move that appears best at the moment without worrying too much about future consequences.





This kind of myopic behavior is simple and convenient, making it an attractive algorithmic strategy

Greedy Algorithm

When a greedy algorithm works: prove its correctness

- Interval scheduling: induction on step
- Optimal loading: induction on input size
- Scheduling to minimum lateness: exchange argument

When a greedy algorithm does not work: find a counterexample

- Coin-changing problem

Outline

1. Introduction to Greedy Algorithms
2. Interval Scheduling
3. Optimal Loading
4. Scheduling to Minimize Maximum Lateness
5. Fractional Knapsack Problem

Interval Scheduling

Input. $S = \{1, 2, \ldots, n\}$ is a set of n jobs, job i starts at s_i and finishes at f_i, i.e., occupies the interval $[s_i, f_i)$.

- Two jobs i and j are compatible if they do not overlap: $s_i \geq f_j$ or $s_j \geq f_i$

Goal: find a maximum-cardinality subset of mutually compatible jobs.

This problem is also known as the unweighted interval scheduling problem or activity selection.

Interval Scheduling

Input. $S = \{1, 2, \ldots, n\}$ is a set of n jobs, job i starts at s_i and finishes at f_i, i.e., occupies the interval $[s_i, f_i)$.

- Two jobs i and j are compatible if they do not overlap: $s_i \geq f_j$ or $s_j \geq f_i$

Goal: find a maximum-cardinality subset of mutually compatible jobs.

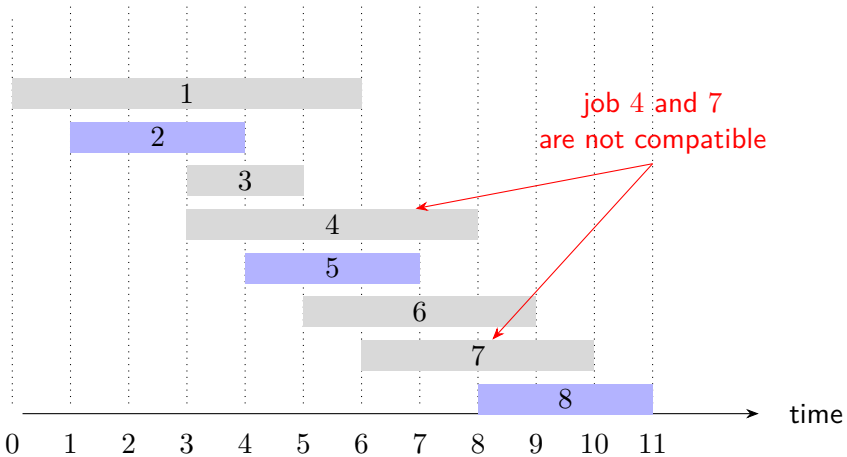
This problem is also known as the unweighted interval scheduling problem or activity selection.

Instance

i	1	2	3	4	5	6	7	8
s_i	0	1	3	3	4	5	6	8
f_i	6	4	5	8	7	9	10	11

Solution. $\{2, 5, 8\}$

Example


1
2
3
4
5
6
7
8
job 4 and 7
are not compatible
time
0 1 2 3 4 5 6 7 8 9 10 11

Interval Scheduling: Greedy Algorithm

Greedy template

- Consider jobs in some natural order, then take each job provided that it is compatible with those already selected.
- Selection strategy is short-sighted $\rightsquigarrow$ the chosen ordering may not be optimal

Interval Scheduling: Greedy Algorithm

Greedy template

- Consider jobs in some natural order, then take each job provided that it is compatible with those already selected.
- Selection strategy is short-sighted $\rightsquigarrow$ the chosen ordering may not be optimal

Candidate selection strategies

- [Earliest start time] Consider jobs in ascending order of s_i
- [Earliest finish time] Consider jobs in ascending order of f_i
- [Shortest interval] Consider jobs in ascending order of $f_i - s_i$
- [Fewest conflicts] For each job j, count the number of conflicting jobs c_j. Schedule in ascending order of c_j.

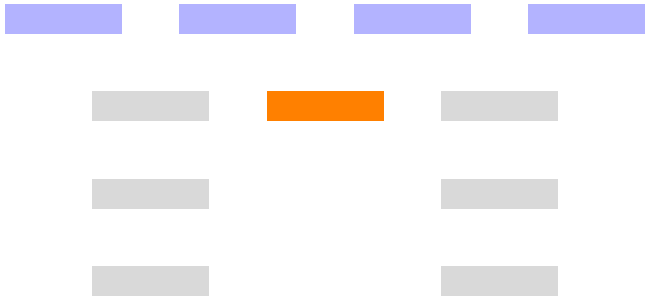
Counterexample for Earliest Start Time



Counterexample for Shortest Interval



Counterexample for Fewest Conflicts



Greedy Algorithm: Earliest-Finish-Time-First

Algorithm 1: GreedySelect$(S, s_i, f_i, i \in [n])$

Output: maximum compatible subset $A \subseteq S$

1: Sort jobs by finish time so that $f_1 \leq \cdots \leq f_n$;
2: $n \leftarrow |S|$;
3: $A \leftarrow \emptyset$;
4: **for** $i \leftarrow 1$ **to** n **do**
5: **if** *job* i *is compatible with* A **then** $A \leftarrow A \cup \{i\}$;
6: **end**
7: **return** A;

Q. How do we determine whether job i is compatible with A?

A. Keep track of the job j^* most recently added to A. Job i is compatible with A iff $s_i \geq f_{j^*}$ holds.

Demo of Earliest Finish Time First

Input. $S = \{1, 2, \ldots, 8\}$

i	1	2	3	4	5	6	7	8
s_i	0	1	3	3	4	5	6	8
f_i	6	4	5	8	7	9	10	11

Solution. $A = \{2, 5, 8\}$

Overall complexity: $\Theta(n \log n)$

- Sorting by finish time: $\Theta(n \log n)$
- Compatibility checking: $O(n)$

Lemma. Earliest-finish-time-first algorithm always gives an optimal solution.

How do we prove it?

Mathematical Induction for Greedy Algorithm

Proof template for greedy algorithm

1. Formulate correctness as a proposition indexed by a natural number n, asserting that the greedy algorithm produces an optimal solution.
   - Here, n could be the number of algorithmic steps or input size.
2. Prove the proposition for all natural numbers.
   - Base case: from the smallest instance
   - Inductive step: type 1 or type 2 induction

Proposition for Earliest-Finish-Time-First

Let S be the job set of size n, s_i and f_i be the start time and finish time.

Proposition. When algorithm GreedySelect carries on the k-th step, it choose k jobs $(i_1 = 1, i_2, \ldots, i_k)$, which is exactly the first k jobs of a maximum compatible subset A of S.

According to the above proposition, $\forall k$, the first k-step choice is exactly the first k-jobs of some maximum compatible subset A, and will yield a maximum compatible subset A in n steps.

Mathematical Induction: Induction Basis

Let $S = \{1, 2, \ldots, n\}$ be the sorted job set: $f_1 \leq \cdots \leq f_n$

Mathematical Induction: Induction Basis

Let $S = \{1, 2, \ldots, n\}$ be the sorted job set: $f_1 \leq \cdots \leq f_n$

Base case. $k = 1$, prove A includes job 1

Mathematical Induction: Induction Basis

Let $S = \{1, 2, \ldots, n\}$ be the sorted job set: $f_1 \leq \cdots \leq f_n$

Base case. $k = 1$, prove A includes job 1

For an arbitrary maximum compatible subset A, sort jobs in A in ascending order according to the finish time.

If the first job in A is j and $j \neq 1$, then replace job j with job 1, yielding A':

$$A' = (A - \{j\}) \cup \{1\}$$

- 1 won't appear in $(A - \{j\}) \Rightarrow |A| = |A'|$
- $f_1 \leq f_j \Rightarrow$ replacement does not affect compatibility $\Rightarrow A'$ is also a maximum compatible subset of S and includes job 1.



Mathematical Induction: Induction Step (1/2)

Assume the proposition holds for k, prove that it also holds for $k+1$

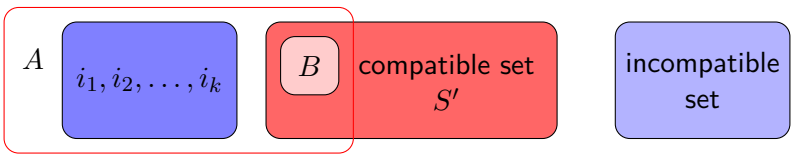
- $(k+1)$-step choice job i_{k+1} and $(i_1, \ldots, i_k)$ forms the first $k+1$ jobs of some A for S.

Proof. After k steps, algorithm chooses $i_1 = 1, i_2, \ldots, i_k$.

Premise $\Rightarrow \exists$ a maximum compatible A that contains $i_1, i_2, \ldots, i_k$.

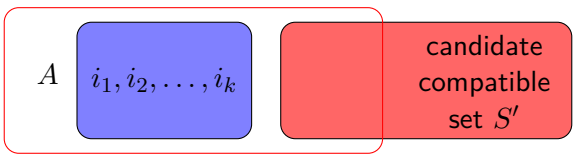
- Let B be the set of other elements in A (already sorted and not empty), and S' be the set of compatible elements w.r.t. $\{i_1, i_2, \ldots, i_k\}$.

$$A = \{i_1, i_2, \ldots, i_k\} \cup B$$
$$S' = \{i \mid i \in S, s_i \geq f_{i_k}\}$$



Mathematical Induction: Induction Step (2/2)

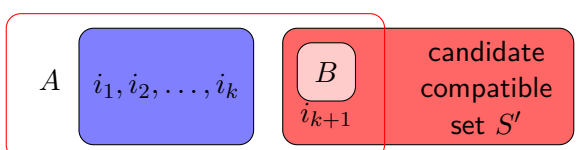
Consider two cases depending on whether job i_{k+1} is the 1st job in B.



Mathematical Induction: Induction Step (2/2)

Consider two cases depending on whether job i_{k+1} is the 1st job in B.

- If i_{k+1} happens to be the first job in B, then the desired result immediately follows, $(k+1)$-step choice still yields the partial solution of A.



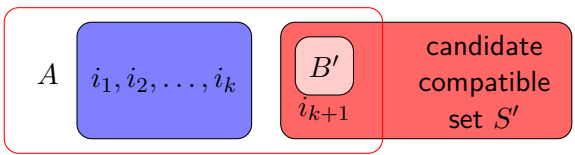
Mathematical Induction: Induction Step (2/2)

Consider two cases depending on whether job i_{k+1} is the 1st job in B.

- If i_{k+1} happens to be the first job in B, then the desired result immediately follows, $(k+1)$-step choice still yields the partial solution of A.
- If i_{k+1} is not the first job in B, then we must have $i_{k+1} \notin B$ (since B is sorted)
  - the greedy selection rule $\Rightarrow$ the finish time of i_{k+1} must be earlier than the first job in B
  - At this point, we can replace the first job in B with job i_{k+1}, yielding B'. Obviously, $|B'| = |B|$.

$$\{i_1, i_2, \ldots, i_k\} \cup B' = A'$$

Note that $|A| = |A'| \Rightarrow A'$ is still a maximum compatible set of S. This proves the induction step.



Outline

1. Introduction to Greedy Algorithms
2. Interval Scheduling
3. Optimal Loading
4. Scheduling to Minimize Maximum Lateness
5. Fractional Knapsack Problem

Optimal Loading Problem

Problem. Given n containers with weights w_i and a boat with maximum weight capacity W (no volume limit).

Goal. A loading plan that maximizes the number of containers on the ship.

Analysis. This problem is a special case of 0-1 knapsack problem.

- item: container
- boat: knapsack
- all $v_i = 1$

Modeling

Let $(x_1, x_2, \ldots, x_n)$ be the solution vector, $x_i \in \{0, 1\}$.

- $x_i = 1$ iff i-th container is on the boat

Objective function:

$$\max \sum_{i=1}^{n} x_i$$

Constraint:

$$\sum_{i=1}^{n} w_i x_i \leq W, x_i \in \{0, 1\}, i \in [n]$$

Algorithm Design

Greedy strategy. lightest first

Algorithm steps

- sort the containers by weight in ascending order, to ensure $w_1 \leq w_2 \leq \cdots \leq w_n$
- load them in this order and stop when loading the next container would exceed the capacity

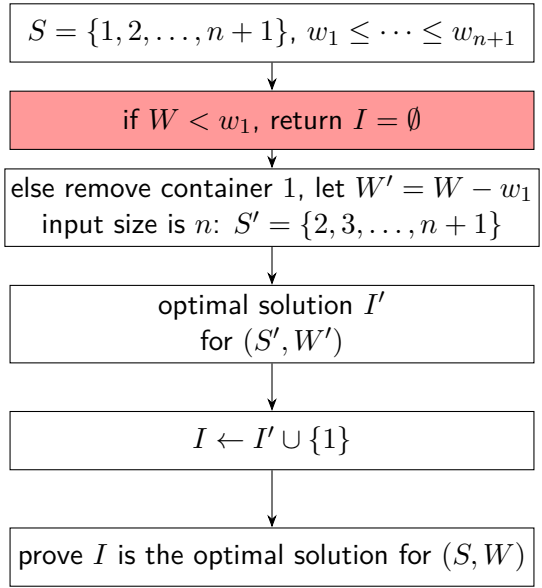
Proof of Correctness (Induction on Input Size)

Lemma. $\forall$ input size n, the algorithm yields the correct solution.

Let $S = \{1, 2, \ldots, n\}$ be the set of containers that have been sorted in ascending order, and $w_1 \leq w_2 \leq \cdots \leq w_n$.

- Base case. Prove when the input size $n = 1$ (there is only one container), the greedy algorithm will yield the correct solution. This obviously holds.
- Inductive step. Prove if the greedy algorithm yields an optimal solution for input size n, it will also yield optimal solution for input size $n + 1$.

Analysis of Greedy Algorithm: Interpretation



Correctness Proof (1/2)

Induction hypothesis: greedy strategy will yield an optimal solution for input size n, consider input size $n+1$

$$S = \{1, 2, \ldots, n+1\}, w_1 \leq w_2 \leq \cdots \leq w_{n+1}$$

Premise of induction $\Rightarrow$ for input size n

$$S' = \{2, \ldots, n+1\}, W' = W - w_1$$

Greedy strategy yields another optimal solution I' for (S', W').

Let $I = I' \cup \{1\}$.

Correctness Proof (2/2)

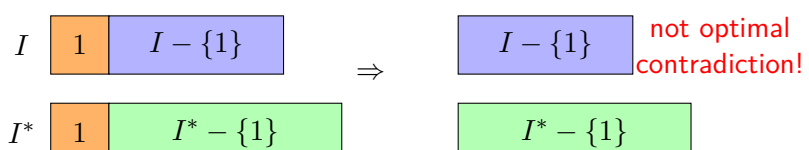
Claim. I is the optimal solution for (S, W).

Proof by contradiction. If not, suppose there exists an optimal solution I^* for (S, W) and $|I^*| > |I|$.

- Assume w.l.o.g. $1 \in I^*$, since otherwise we can replace the first container in I^* with 1, also yielding the optimal solution.
- $I^* - \{1\}$ forms a solution for (S', W') and

$$|I^* - \{1\}| > |I - \{1\}| = |I'|$$

The existence of I^* contradicts the hypothesis that I' is the optimal solution for (S', W').



Summary

0-1 knapsack is an $\mathcal{NP}$-hard problem

- optimal loading is a special case of 0-1 knapsack problem, and can be solved efficiently using a greedy algorithm

Correctness proof. Induction on input size

Outline

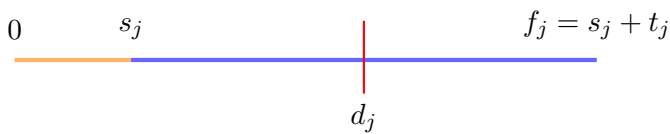
1. Introduction to Greedy Algorithms
2. Interval Scheduling
3. Optimal Loading
4. Scheduling to Minimize Maximum Lateness
5. Fractional Knapsack Problem

Scheduling to Minimize Maximum Lateness

Minimizing maximum lateness problem (最小最大延迟调度)

- Given a set A of jobs, a single machine processes one job at a time, and all jobs are released at time 0
- Job j requires t_j units of processing time and is due at time d_j (deadline).
- If job j starts at time s_j, it finishes at time $f_j = s_j + t_j$.
- Scheduling: $S : A \to \mathbb{N}$, $S(j) = s_j$ is the start time of job j.
- Lateness: Lateness function L_S computes the lateness of job under schedule S:

$$L_S(j) = \ell_j = f_j - d_j = s_j + t_j - d_j, \text{ which may be negative.}$$



Modeling of Minimizing Maximum Lateness

Goal. Schedule all jobs to minimize maximum lateness

$$\min_S\{\max_{j\in A} L_S(j)\} := \min_S\{\max_{j\in A} \ell_j\} = \min_S\{\max_{j\in A}\{s_j + t_j - d_j\}\}$$



Constraint. No overlap

$$\forall i, j \in A, i \neq j$$
$$s_i + t_i \leq s_j \vee s_j + t_j \leq s_i$$

Example 1

A	1	2	3	4	5
S	0	5	13	17	27
T	5	8	4	10	3
D	10	12	15	11	20
L	0	1	2	16	10

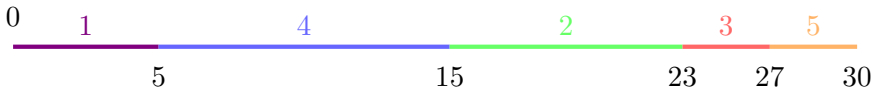
Table: Sequential scheduling



Example 2

A	1	4	2	3	5
S	0	5	15	23	27
T	5	10	8	4	3
D	10	11	12	15	20
L	0	4	11	12	10

Table: Earliest-deadline first



Minimizing Maximum Lateness: Greedy Algorithm

Greedy template. Schedule jobs according to some natural order.

- [Shortest processing time first] Schedule jobs in ascending order of processing time t_j.

A	1	2
T	1	10
D	100	10

- $\ell_1 = 0$, $\ell_2 = 11 - 10 = 1$
- $\ell_2 = 0$, $\ell_1 = 0$ (better)

- [Smallest slack] Schedule jobs in ascending order of slack $d_j - t_j$.

A	1	2
T	1	10
D	2	10

- $\ell_2 = 10 - 10 = 0$, $\ell_1 = 11 - 2 = 9$
- $\ell_1 = 0$, $\ell_2 = 10 + 1 - 10 = 1$ (better)

Minimizing Maximum Lateness: Earliest Deadline First

Algorithm 2: Schedule(A, T, D)

1: sort n jobs in A so that $d_1 \leq d_2 \leq \cdots \leq d_n$;
2: $t \leftarrow 0$; `// from time 0`
3: **for** $j = 1$ *to* n **do**
4: assign job j to interval $[t, t + t_j)$;
5: $s_j \leftarrow t$;
6: $f_j \leftarrow t + t_j$;
7: $t \leftarrow t + t_j$;
8: **end**
9: **return** *intervals* $[s_1, f_1), \ldots, [s_n, f_n)$

Main idea

- Earliest deadline first
- Schedule jobs consecutively with no idle time

Correctness Proof: Exchange Argument

Proof sketch

- Analyze the difference between optimal solution and algorithm solution (e.g. different order)
- Design a transform operation (e.g. swap), thus we can gradually transform an optimal schedule to the greedy algorithm schedule in finite steps.
- The transformation does not affect optimality of solution, since every step preserves optimality.

In this case, two properties of greedy algorithm schedule:

- No idle time: every time there is a job being processed
- No inversion. We say (i, j) forms an inversion if $d_i > d_j$ but $s_i < s_j$

Key Lemma about Algorithm Solution

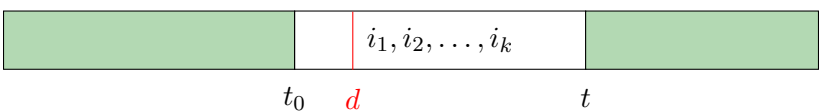
Lemma. All schedules with no inversions and no idle time have the same maximum lateness.

Proof. No inversion $\Rightarrow$ tasks are sorted in ascending order of d_i.

It is possible that several jobs have the same deadline. Jobs $i_1, i_2, \ldots, i_k$ with the same deadline d are assigned arbitrarily. (green parts are identical)

- The start time is t_0, the finish time of the last job in the equal-deadline block is t, among these jobs, the max lateness is $\max\{0, t - d\}$ $\Leftarrow$ independent of the order of $i_1, i_2, \ldots, i_k$.

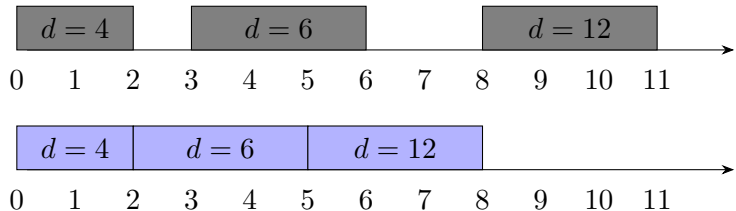
$$t = t_0 + (t_{i_1} + t_{i_2} + \cdots + t_{i_k})$$



Corollary. All possible algorithm solutions have the same maximum lateness time.

Examine the Optimal Solution

Observation. There always exists an optimal schedule with no idle time.



Algorithm solution: the earliest-deadline-first schedule has no idle time.

- We have eliminated one difference between an optimal schedule and the greedy algorithm schedule.
- The remaining difference is the presence of inversions.

Minimizing Lateness: Inversions

Inversion. Given a schedule S, an inversion is a pair of jobs i and j such that $d_i < d_j$ but job j is scheduled before job i, i.e., $s_j < s_i$.

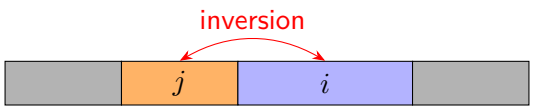
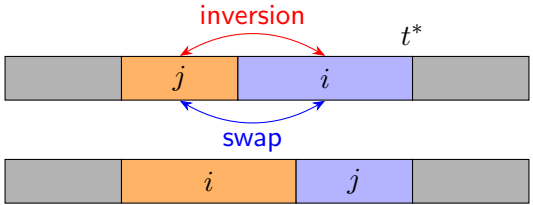


Figure: As before, jobs are numbered so that $d_1 \leq d_2 \leq \cdots \leq d_n$

Fact. If a schedule (with no idle time) has an inversion, it has at least one pair of inverted jobs scheduled consecutively.

Minimizing Maximum Lateness: Inversions

Claim. Swapping two adjacent, inverted jobs reduces the number of inversions by one and does not increase the max lateness.



Proof. Let ℓ be the maximum lateness before the swap and ℓ' the maximum lateness afterward.

- $i \leftrightarrow j$ does not affect the lateness of the other jobs: $\ell'_k = \ell_k$ for all $k \neq i, j$
- $\ell'_i \leq \ell_i$ (because job i has been moved earlier)
- $\ell'_j = t^* - d_j$ (definition), i and j are inverted $\Rightarrow d_i < d_j$, thus $\ell'_j \leq t^* - d_i = \ell_i$

$$\Rightarrow \max\{\ell_i, \ell_j\} \geq \max\{\ell'_i, \ell'_j\}$$

Putting All the Above Together

Theorem. The earliest-deadline-first schedule S is optimal.

Proof. Define S^* to be an optimal schedule. Let us see what happens.

- Can always assume S^* has no idle time.
- If S^* has no inversions, then key lemma $S \sim S^*$, stop here.
- If S^* has an inversion, let $i \leftrightarrow j$ be an adjacent inversion. Swapping i and j:
  - does not increase the max lateness
  - strictly decreases the number of inversions
- Continue the above process until there is no inversion; we can also conclude that $S \sim S^*$.

The maximum possible number of inversions is $n(n-1)/2$ (completely inverted), thus the transformation will stop in finite steps.

Summary of Greedy Analysis Trick

Analysis. Find the difference between optimal solution and algorithm solution.

Exchange argument. Gradually transform an optimal solution into the greedy solution through:

- a finite sequence of steps
- each exchange preserves solution quality

Outline

1. Introduction to Greedy Algorithms
2. Interval Scheduling
3. Optimal Loading
4. Scheduling to Minimize Maximum Lateness
5. Fractional Knapsack Problem

Fractional Knapsack Problem

Input. Given n items with weight vector $(w_1, \ldots, w_n)$ and value vector $(v_1, \ldots, v_n)$, and weight limit $W > 0$.

Goal. Find $x = (p_1, \ldots, p_n) \in [0, 1]^n$ (choose some fractions of n items) to satisfy:

- Objective: maximizes $\sum_{i=1}^{n} p_i v_i$
- Constraint: $\sum_{i=1}^{n} p_i w_i \leq W$

The difference is that now the items are infinitely divisible.

Greedy Algorithm

Greedy strategy. highest value-to-weight ratio first

Algorithm

- Sort the items in descending order of value-to-weight ratio $\alpha_i = v_i / w_i$.
- Iteratively select the item with the greatest value-to-weight ratio
- If the next item does not fit completely, take the fraction that fills the remaining capacity.

Correctness Proof (1/3)

Lemma. $\forall$ input size n, the algorithm yields the optimal solution.

Proof idea. Mathematical induction on input size.

Base case. When $n = 1$, the greedy algorithm produces an optimal solution.

Inductive step. Suppose the algorithm is optimal for $n = k$, then it is also optimal for $n = k + 1$.

- Let p_1 be the algorithm's output for the first item, $I' = (p_2, \ldots, p_{k+1})$ be the output on instance $(w_2, \ldots, w_{k+1})$, $(v_2, \ldots, v_{k+1})$, and $W - p_1 w_1$.
- According to the induction premise, I' is the optimal solution of the above sub-instance of size $n = k$. Let $I = p_1 \cup I'$.

Claim. I is the optimal solution for $n = k + 1$.

Correctness Proof (2/3)

Proof by contradiction. If not, there must exist a better solution I^* with maximal value V^*.

Prove the first element p_1^* of I^* must be equal to p_1 of I.

1. $p_1^* = p_1$: we have nothing to prove.
2. $p_1^* > p_1$ is impossible, because the greedy strategy guarantees that p_1 of I is as large as possible.
3. If $p_1^* < p_1$, we can always increase it to p_1 by decreasing total weight of its remaining k items by $\Delta = (p_1 - p_1^*)w_1$. Note that such adjustment makes sense since the total weight of the remaining k items must be larger than Δ. Otherwise, we must have $V^* < V$, which is not possible by premise. We then consider two sub-cases after adjustment:
   - The total value is unchanged. This is only possible when there exists at least one more item j such that $\alpha_j = \alpha_1$.
   - The total value is higher. However, this case will never occur since it goes against the assumed optimality of I^*.

Correctness Proof (3/3)

We conclude that either $p_1^* = p_1$ or we can adjust it to this case without compromising optimality.

$I^* - \{p_1\}$ forms a solution for $W - p_1^* w_1 = W - p_1 w_1$ with items $(2, \ldots, n+1)$ with total value $V^* - p_1 v_1 > V - p_1 v_1 \rightsquigarrow$ contradicts the optimality of I'

This proves that I is the optimal solution for input size $n = k + 1$.